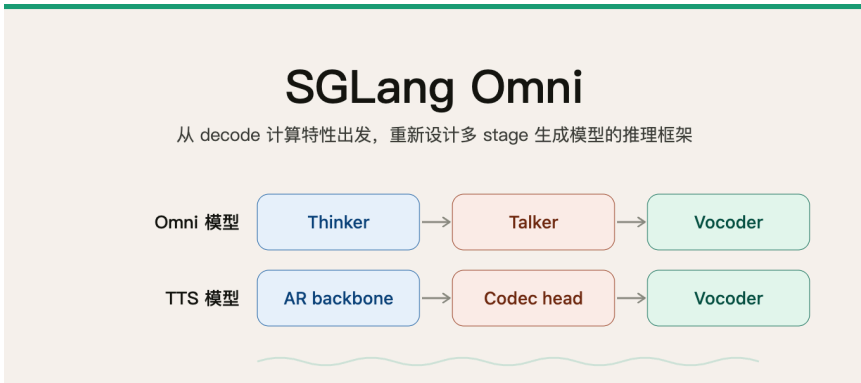




SGLang Omni：从 decode 计算特性出发，重新设计多 stage 生成模型的推理框架



Chayenne Zhao

临战弹琴，遇事摸鱼，赶 paper 不急

已关注

BBuf 等 200 人赞同了该文章

从今年 2 月底开始，我个人的工作重心从 SGLang RL 移到 SGLang Omni。坦诚说，在工作的一开始，我并没有多少底气。熟悉我的人都知道，我并不是系统方向出身，甚至在很长一段时间里，对系统类的工作有一种本能的畏惧。我曾经在朋友圈里写过这样一段话：

犹记得本科低年级时，我对系统课程有着极深的抵触。入学第一个学期被动态规划折磨到觉得自己在计算机这一行毫无可能，第二年在计算机系统概论课上的理解困难，更让我对系统的庞大学问产生了一种近乎逃避的心理。恰好那时接触了粗浅的人工智能课程，在玩具工程上找到了一点投机取巧式的成就感，便天真地以为自己可以绕开所有系统，去做那些看起来更“智能”的事情。

直到大学三年级，我在美国暑研期间，才在导师和指导我的博士生的带领下，通过我人生的第一个开源项目，开始理解从硬件到软件，系统的方方面面。计算机系统是一种思维，是时时刻刻都在思考如何让自己的工作被他人认可，简单易用，能扩展到更大规模，带来更大影响。由衷感谢那个暑假指导我的导师 Graham 和 Sherry，还有耐心负责的 PhD Vijay。让我第一次意识到了系统之美。

而意识到系统的强大，则一直等到了本科毕业之后。坦诚说，知道了系统的美妙之处后，我本科的系统学习也并没有付诸更多心思。我至今提起都会觉得羞愧的是，我的大多数硬核系统课程，譬如计算机组成原理、计算机网络原理、操作系统，这些都是在我绩点自由的大四修完的。若干年后想起来，每天都在和这些基本功打交道的我，其实这些基本功都没有好好学过。我都还记得本科毕业前其实可以选择章明星老师的一门选修课，但是那课对我当时的状态实在是不能完成的。一年后加上章老师微信，我都惭愧到不好意思给他提起这件事情。

正是这段不光彩的本科生涯，让我在踏入系统研究之后，始终带着一份小心翼翼的自省。我很清楚，自己在基本功上有所欠缺，基本每向前走一步，都需要去补上过去欠下的功课。事实也是如此，从 SGLang RL 到 SGLang Omni，我们处理的问题看上去完全改变了。以前我们会去处理显存的 offload 和 upload，研究怎么高效地在训练引擎和推理引擎中间去传递参数。现在，对 Omni，我们考虑不同 stage 之间的 fusion，思考如何用一张统一的 **CUDA Graph** 去覆盖整个 decoding 过程。这是一个非常有趣的变化，我几乎开始从事和以前完全不相关的内容，以至于有时候和朋友提起，我去做 SGLang Omni 了，总会有人诧异，怎么开始做了完全不相关的事情。

对此，我的想法是，一方面，我很早前就分享过 SGLang RL 小组的早期成员 Junrong 的观点：“我们 RL 系统的优化往往 block 在我们对推理系统本身的认知”，所以进一步去理解推理系统，对我喜欢的 RL 工作本身，也大有裨益；另一方面，Omni 模型本身也存在 RL 需求，譬如 TTS 和 Qwen3 Omni 模型的 Thinker，这些组件都具有明确的 reward 目标。

▲ 赞同 200 ▼ ● 4 条评论 ★ 203 ♥ 13 ➦ 分享 📄 申请转载 ✨ ...

关于作者



Chayenne Zhao

临战弹琴，遇事摸鱼，赶 p...

银光、陈乐群、菽陌松围也关注了他

回答

31

文章

77

关注者

9,654

已关注

发私信

大家都在搜

换一换

- 阿根廷 3-1 瑞士进 4 强 357 万 新
- 6 月新增新冠确诊 7.9 万例 357 万 新
- 台风巴威深入内陆 356 万 新
- 长征十号乙火箭回收成功 356 万
- 台风巴威登陆 347 万 热
- 哈兰德告别世界杯 333 万 新
- 长沙通报体育局员工恶意... 319 万 热
- 晋江鞋厂火灾已致 28 死 300 万
- 隋坡差评餐厅老板逆天回怼 294 万 热
- 鹿晗 司晓迪 293 万 热

的计算和推理设计更合理的结构, 并在此基础上做微妙的推理引擎调度优化。SGLang Omni 所面对的模型虽然和 LLM 差别显著, 但解码阶段的核心计算特性高度相通, 用到的优化思路也常常殊途同归。在我看来, 机器学习系统研究者只有一个目标, 研究指定计算过程的计算特性, 并且针对其计算特性设计高效鲁棒的系统。

因此, 这篇文章既是我们 SGLang Omni 项目组对当前技术框架和阶段性工作的系统总结, 也是我们想认真回答几个根本问题:

1. 我们要优化的是怎样的计算过程?
2. 这个计算过程具有怎样的计算特性?
3. 我们为此设计了怎样的系统?
4. 我们期待 SGLang Omni 走向何处?

此外, 幸运的是, 这次我仍旧不是一个人在探索。从 SGLang RL 社区开始, 我们有了越来越多的伙伴, 尽管大家可能因为工作和生活的变故, 并不会永远坚守在开源社区的一线, 但是社区强大的凝聚力和打满每一张 GPU 的热情, 永远在此。这一次, 我还是幸运地遇见了同样优秀的伙伴们, 我们一同来重新审视 Omni 模型的推理过程, 并为此设计扎根于计算特性的推理框架。SGLang Omni 还在热烈的开发过程中, 我们也永远欢迎和我们志同道合, 对机器学习系统有着近乎偏执的美学追求的朋友加入我们。

致谢 (按照姓氏拼音排序):

Ke Bai, Haoguang Cai, Shangming Cai, Qiujiang Chen, Chen Cheng, Jiaxin Deng, Wenyaoyao Gao, Yifei Gao, Jingwen Gu, Yitong Guan, Chenchen Hong, Hao Jin, Xinli Jing, Shenggui Li, Junrong Lin, Xinyuan Liu, Xinyu Lu, Yuan Luo, Silin Men, Ratish P, Chengliang Qian, Jinjiang Qu, Dongming Sheng, Shuai Shi, Chao Wang, Richard Wang, Shuwen Wang, Zijie Xia, Yuhao Yang, Xuesong Ye, Fan Yin, Gaokai Zhang, Xiaoyu Zhang, Yichi Zhang, Huapeng Zhou, Chenyang Zhao

我们要优化怎样的计算过程?

早在去年 4 月份, Qwen 团队第一次发布 Qwen 2.5 Omni 的时候, 我们就尝试支持过 Qwen 2.5 Omni 在 SGLang Main 中。从模态上来感受, Qwen 2.5 VL 和 Qwen 2.5 Omni 的输入是非常类似的, 可以理解为, Qwen 2.5 Omni 的 thinker 就是在 Qwen 2.5 VL 的基础上增加了音频输入。逻辑上, 抛开 audio encoder 采用的具体技术路线, 可以抽象理解为 Qwen 2.5 Omni 的 thinker 就是更强大的、更多模态的 Qwen 2.5 VL。但是很遗憾, Omni 模型的技术路线不单单如此, 这里又涉及到另一个问题, Omni 模型的定义尚不统一。

在部分模型团队的定义中, Omni Model 可以算作支持音频输入的 VLM, 比如 [MiMo Omni](#) 和 [Nemotron Omni](#)。而在 Qwen Omni 团队的定义中, Omni Model 不单单支持音频输入, 更支持音频输出。我经常举的一个例子其实是 GPT-4o 和 Google 的 Gemini App。例如, 假设我们在超市里面买菜, 打开 Gemini App 的摄像头模式, 这些模型一边接受着摄像头里面的视频输入, 与此同时, 它们还接受我们的语音输入。而输出端, 我们能够看到屏幕上出现文字输出, 并且也有对应的语音输出。从输入输出模态上, Qwen 所代表的 Omni 系列支持「语音 + 文字 + 视频 + 图片」的输入, 并且支持「文字 + 语音」的输出。这样的 Omni 定义其实也非常常见, 譬如特斯拉上的 Grok APP 和豆包的手机 APP。涵盖面最广的 Omni 模型仍不止于此, 来自蚂蚁团队的 [Ming Omni](#) 模型, 做到了全模态的输入和输出, 为了能够兼容文字和图片的输出, 其输出部分同时支持 auto regressive decoding 和 denoising/diffusion。除此之外, 还有素来与 SGLang 团队合作极多的 LLaDA 团队推出的 [LLaDA Uni](#) 模型, 和 Ming Omni 一样拥有令人惊叹的全模态能力。

对于如此多种的 Omni 模型定义, 我自己并无偏好, 各家模型团队都为此付出了巨大的努力, 每一种定义在各自的语境下、对算法研究者而言都成立。只是对我们做推理系统的人来说, 按输入输出模态来划分不是一个特别合理的视角——模态告诉我们模型对外的封装是什么样, 却没有直接表现出它的计算过程。如同我提到的那样, 机器学习系统研究关注计算过程

而非候坐的好坏——下面提到的每一「候坐」，背后都是一家团队攻关山杉的工夫，我们在此不是选择沿着计算过程的维度，用 multi-stage decoding 的方式来进行分类。

我们以 Qwen3-Omni⁺ 为例，来解释 multi-stage decoding 的大致流程（详细过程可以参考我之前的博客）。Qwen3-Omni 生成一条同时带文字和语音的输出，不是一个 decode loop 自回归地从头到尾，而是几个异构的 decode stage 的交替：Thinker 自回归地生成文字，Talker 自回归地生成每个 timestep 的第 0 个 codec token，再由 MTP 补全该 timestep 余下的 codec token，几个 decode stage 构成异步流水线。在 SGLang Omni 的设计理念里，Qwen3-Omni 的关键不在于多模态，而在于 end 2 end decoding 被拆成了计算过程各异的多个 stage，这就是 multi-stage decoding。

有趣的是，multi-stage decoding 并不是 Omni 模型的专利，许多 TTS 模型同样采用了 multi-stage 的 decoding 过程。比如 FishAudio 的 S2 Pro，用一种被称为 Dual-AR 的双段流水来生成语音：一个约 4B 的 Slow AR 沿时间轴逐帧生成语义 token，一个约 400M 的 Fast AR 在每帧内补全声学 codec token，两个 stage 串行嵌套。它们没有理解多模态输入的使命，但 decoding 一侧的 multi-stage 结构，和 Qwen3-Omni 异曲同工。

从“decoding 是否 multi-stage”的视角出发，各个团队的模型自然地落在两侧，各自有最适合它们的推理方式。

类型	代表模型	decoding 形态	是否属于 SGLang Omni 的设计目标
Single-stage (AR)	MiMo Omni、Nemotron Omni、Qwen ASR、MiMo ASR	单条标准 LLM/VLM decode loop	否——SGLang 主线已能极致优化
Single-stage (diffusion)	Wan、Qwen-Image	单条 denoising loop	否——SGLang Diffusion 已支持
Multi-stage	Qwen3-Omni、Fish S2 Pro、Qwen3-TTS、Voxtral、Higgs、Ming Omni、LLaDA Uni	多个异构 decode stage 串/并联	是

一侧是 single-stage decoding。MiMo Omni、Nemotron Omni 这样的模型，能理解音频、视频、图片等丰富的输入，输出则为文本，decoding 就是一条标准的 LLM / VLM decode loop；Qwen ASR、MiMo ASR 接收音频、输出文字转写，同样如此。这些都是非常强大的模型，而它们的 decoding 恰好是 SGLang 主线最擅长的形态——SGLang 本身已经能把它们的推理做到极致，并不额外需要排布 multi-stage pipeline 这一层。

而另一侧是 multi-stage decoding，SGLang Omni 想要补全的拼图。Qwen3-Omni、许多 TTS 模型（S2 Pro、Qwen3-TTS、Voxtral、Higgs 等），以及做到全模态输出的 Ming Omni、LLaDA Uni，它们的 decoding 都被拆成了多个异构 stage。这些模型同样凝结着众多团队令人敬佩的工作，它们共有的那套 multi-stage 计算过程，正是我们设计 SGLang Omni 的初衷。

无独有偶，diffusion 模型也能按此分类清楚。像 Wan、Qwen-Image 这样出色的开源图像生成模型，它们的 decoding 是单独一条 denoising loop，没有第二个 decode stage 接力，属于 single-stage；SGLang Diffusion 已经有专门面向 diffusion 的领先推理方案，所以它们不在 SGLang Omni 的设计目标中，哪怕“输出图像”这件事听上去很 Omni。反之，Ming Omni 里也用到了 diffusion，但 Ming 的 diffusion block 只是其 multi-stage 流水线中的一个 stage（AR 骨干先生成，再分流到 audio decoder 或 diffusion image decoder），所以 Ming Omni 是 multi-stage 的，是 SGLang Omni 的目标。在 SGLang Omni 依照计算过程而划分的设计理念中，决定一个模型的归属，不是用没用 diffusion，而是 decoding 是不是 multi-stage。同样是图像生成模型，在纯图像生成模型里它就是全部，在 Ming Omni 里它只是一个

completion 这样的多 stage, 天生自回归和 multi-stage 高度相关。但我始终认为, 计算过程才是机器学习系统设计的本质出发点。机器学习系统研究者与算法研究者可以有着不同的美学追求。

Multi-Stage Decoding 模型的计算特性如何?

上一节确立了 SGLang Omni 真正想服务的对象——decoding 被拆成多个异构 stage 的模型。一个自然的问题是, 这些模型在计算上究竟有什么共性? 这些共性又将如何决定我们做推理系统时的取舍?

我们可以从一个具体的例子出发, 考虑 Qwen 2.5 VL 和 Qwen 2.5 Omni Thinker-Talker 的差异: 对于 Thinker 而言, 问题相对清晰, 可以定义为在 VLM 之外多增加了一个音频 encoder。但是, 将 Talker 纳入考虑后, Thinker 和 Talker 的计算过程, 从内存访问模式、调度节奏到对延迟的敏感度都有明显差距。

Thinker 是标准的自回归 decode loop, 计算瓶颈在 attention 和 KV cache 管理, 目标是同时抬高 TPOT 和 Throughput。SGLang 主线对此已经积累了成熟的 prefill/decode 分离、chunked prefill、continuous batching 等一整套方案。而 Talker 情况有所不同, Talker 本身是一个自回归 backbone, 每步生成当前 timestep 的第 0 个 codec token; 紧接着 MTP 以 Talker 的首 codec token 为条件, 并行补全该 timestep 余下的 codec token, 并把补全结果的 embedding 回写给 Talker 作为下一步的输入条件。这里的反馈回路不在 Thinker 和 Talker 之间, 而在 Talker 和 MTP 之间。MTP 的输出结果写回到 Talker buffer, Talker 下一步消费。这导致 Talker 单步的计算不再是大矩阵乘主导, 而是 backbone 的一次轻量 forward 加上 MTP 的 multi-head 补全和 embedding 回写交替。此外, Talker 对延迟的要求比 Thinker 更苛刻——用户听到第一个音节取决于 Talker 多快走完第一个 timestep 的 backbone + MTP, 但单步计算量远小于 Thinker, GPU 利用率天然就低。因此, 我们可以将 Thinker 的 prefill 过程理解为 compute bound, Thinker 的 decode 过程理解为 memory bound。这两类 bound 在整个机器学习系统中被广泛研究: prefill 阶段 GPU 的算力是瓶颈, 要尽可能把所有矩阵乘打满; decode 阶段显存带宽是瓶颈, 每一次从 KV cache 里读出一整串历史 key/value 的开销远大于做一次新的矩阵乘, 所以 decode 优化的核心在 KV cache 管理和 memory access pattern 的压缩。

至于 Talker 与 MTP, 情况更复杂且非标准化。Talker 的 backbone 是自回归的, 看起来像一个小 decode loop, 但它每一步不读长序列 KV cache, 输入只有当前步的 Thinker embedding 和 MTP 反馈的 codec embedding, attention 计算极轻, 显存带宽远不是瓶颈——不是 memory bound。MTP 的补全像微型 prefill, 但每次只处理个位数的 codec token, 计算量太小, 也远碰不到算力天花板——不是 compute bound。两者串在一起, 真正的特点是延迟敏感、步间反馈依赖极紧、单步操作极轻, kernel launch overhead 和同步开销反而成了主要矛盾。

计算范式异构

把这三种计算范式摊在一起看, multi-stage decoding 的第一个共性就很清楚了: 各 stage 的计算密集度、内存访问模式和延迟容忍度高度异构。如果直接在 SGLang 主仓现有的单一 Scheduler 里实现这个过程, compute bound、memory bound、以及两者皆非的延迟敏感型计算, 这三种不同范式被硬拼在一条端到端流水线上, Thinker 的吞吐会被 Talker 的细碎操作打断, Talker 的延迟又会被 Thinker 的大 batch prefill 拖慢。异构意味着解耦不是可选项, 是必然; Thinker 和 Talker 只能通过两个异步的 SGLang Scheduler 异步调度。

依赖模式分化

异构带来的不只是调度上的解耦诉求——stage 之间如何传递数据, 同样需要分情况讨论。第二个共性是 stage 之间的依赖模式出现了分化。LLM 推理是单向的生产者-消费者: prefill 产出 KV cache, decode 消费。但 multi-stage 场景下至少存在两种数据依赖关系。Thinker 和

前需要更低开销的流式缓存且核心延迟低, 后者需要至少内存延迟低, 尽力降低调度开销。

显存争用

第三个计算特性是显存管理。单 Scheduler 管理模型推理非常清楚: 模型权重占据部分 GPU, 余下交给 KV cache (详细情况可见[本人先前的博客](#))。但 multi-stage 场景下, Thinker 和 Talker 的权重都要常驻显存, Thinker 需要维护自身的 prefix cache, 各类 encoder 需要为了 long sequence 预留显存, Talker 需要维护自身与 MTP 之间的反馈 buffer, Talker 还要维护自身的 KV Cache...问题不单单是分蛋糕的玩家多了, 而且不同 stage 的加载顺序不同, offload 策略不同, 每一步的剩余可用显存各有所指, 单个 SGLang scheduler 的 memory fraction 语义在 multi-stage 下需要被重新定义, 我们需要重新构建一整套跨 stage 的显存分配机制。

将这些计算特性一同考虑, multi-stage decoding 对推理框架的需求逐渐明朗起来。异构 stage 的调度与执行需要得到优雅的解耦, 需要建设一套开销可控的跨 stage 流式通信机制, 并且在显存和计算资源上做精细的预算与隔离。这些需求对 SGLang 本身存在极大的侵入性, 但我们将每个 stage 封装为一个 SGLang Scheduler, 情况豁然开朗。我们需要从这些计算特性出发, 重新设计一套完整的 SGLang Omni 架构。

我们设计了怎样的系统?

上一节末尾点出了三个具体诉求——异构 stage 的调度需要优雅解耦、跨 stage 的通信需要开销可控、跨 stage 的显存需要精细预算。这一节我们逐项落地, 让每个 stage 按自己最擅长的方式做计算, 尽力降低其通讯开销, 并且在显存和计算资源上做精细的预算与隔离。

调度解耦

如同先前的描述, 调度上, 我们需要避免 compute bound、memory bound、以及延迟敏感的轻量串行执行这三种范式在复杂循环内互相拖累。方法已经很清晰了, 复用 SGLang Scheduler 为单个 stage 已有的调度设计, 但是为每个 stage 封装一个独立的 Scheduler, 各自执行自身的调度循环。

对于 Thinker 这样的 AR stage, 我们直接复用 SGLang Scheduler 的成熟调度能力——[OmniScheduler⁺](#) 保留了 continuous batching、prefill/decode 混合调度、KV cache 管理、tree cache、overlap scheduling 等全部关键能力, 去掉 tokenizer、grammar、speculative decoding 等 omni 场景目前不需要的模块。对于 preprocessing、encoder 这类不需要调度的 stage, SimpleScheduler 就是一个直白的 get → forward → put 循环。对于 vocoder 这类流式 stage, Code2WavScheduler 则负责维护 per-request 的累积状态, 按窗口解码并输出音频帧。

Talker 的调度是最有趣的问题。Talker 本身是一个 AR 模型, 沿时间轴逐帧生成 codec token, 有自己的 paged KV cache 管理需求, 因此它同样运行在 OmniScheduler 之上, 与 Thinker 各自维护独立的调度循环, 两者通过 relay 异步解耦。但 Talker 与 MTP 之间是同步耦合的, Talker 每产生一个第 0 个 codec token, MTP 必须立刻补全当前 timestep 的剩余 token 并将 embedding 回写, Talker 的下一步严格依赖这次回写。如果把 Talker 和 MTP 拆成两个 Stage、中间再走一趟 relay 和 ZMQ 信号, 单步延迟会显著增大。我们把 Talker 和 MTP 放在同一个 Stage 内, MTP 的补全与反馈回写全部封装在 FeedbackARModelRunner 的一次 forward 调用中完成, 对更上层的 Coordinator 而言 Talker 和 MTP 的一个 timestep 只是一次轻量的 decode step, 完全感知不到 MTP 的存在。需要强调的是, 合并到同一个 Stage 改变的只是 kernel 的编排顺序与 CUDA Graph 的边界, Talker 自身的 paged KV cache、MTP 自身的权重与多头补全逻辑都没有被改变——两者依旧是两个完整的模型, 只是共享了同一次 forward 调用。Talker 和 MTP 之间的紧密反馈回路被限制在 Stage 内部, 没有跨调度器的开销, kernel launch 和同步也可以被 piecewise CUDA Graph 拍平, 这正是延迟敏感的低计算密度串行任务的最佳方案。

用, 也为未来更多 stage 交互由山 J 下/的/ 展路径。

通信分层

为了满足各个 stage 之间的高效通信, 我们将通迅过程拆解为了控制面和数据面。控制消息, 例如“新请求来了”、“上游 chunk 已写入”、“中止请求”, 走 ZMQ, 轻量、成熟、不易出错。而实际的数据面, 大块张量走 relay, 同机 GPU 之间用共享内存或 CUDA IPC 做零拷贝, 跨节点走 NCCL 或 RDMA。

分层的通讯策略也对应了前文分析的两类依赖模式。Thinker 和 Talker 之间的异步解耦, Thinker 把 token 和 hidden state 写入 relay, 发一个 DataReady 信号, Talker 按自己的节奏消费, 两者各自维护独立的 decode loop, 不需要步调一致。而 Talker 与 MTP 之间则是同步紧耦合, MTP 必须等 Talker 产出第 0 个 token, Talker 下一步必须等 MTP 回写。这种内部反馈回路被关在同一个 Stage 的 ModelRunner 里, 不需要 stage 间通信, 避免了额外调度开销。

最后, 在各个 stage 之上, Coordinator 完成高层协调, 新请求路由并发送给入口 Stage, 收集 terminal Stage 的输出, 多个 terminal 同时有结果时负责合并。它并不因为模型细节而特化, 唯一服务于 pipeline 拓扑。

显存隔离

multi-stage decoding 场景与单个 Scheduler 管理的 single stage decoding 在显存配置上差异明显。我们将显存分配从“单一 stage 的全局比例”升级为“跨 stage 的预算机制”。每个 stage 在配置中声明自己的 total_gpu_memory_fraction, 启动时系统按 GPU 维度求和校验——超出卡容量直接拒绝, 不超出就为 AR stage 尽力分配 KV cache 预设空间。于是同一资源组上的 Thinker 和 Talker 各自有自己的显存预算, encoder 的激活峰值也被限制在声明的额度内, 能够为长序列留下充分的显存空间。

在显存管理上, encoder 是 Omni 模型里最容易被低估的风险点。Qwen3-Omni 的 vision encoder 和 audio encoder 权重合计不过 2.5 GB, 并不大——真正的压力来自激活: 长音频或视频输入会把 encoder 的激活峰值推到单卡根本无法承受的程度, 一分钟视频轻易超过 30 GB。因此, encoder 不是附带的小模块, 而是显存预算里必须被严肃对待的一等公民。幸运的是, SGLang Omni 的架构设计对这件事的承接非常自然。encoder 和其他 stage 一样, 在 StageConfig 里声明 tp_size 和 GPU 放置, 由 MultiProcessRunner 统一管理进程、分配 NCCL 端口、协调 rank 间同步。TP 切分激活峰值的能力不是为 encoder 而单独配置, 本身就对所有 stage 成立。TP 机制在每个 stage 内的良好复用, 恰好体现了 SGLang Omni 在 stage 抽象和 placement 上的扩展灵活性。

我们期待怎样的 SGLang Omni?

写到这里, SGLang Omni 项目的整体设计呼之欲出。我们也非常自豪, 目前的 SGLang Omni 已经初具我们团队审美的样子, 正在热烈生长。我们相信 SGLang Omni 架构的生命力, 我们从 Qwen3-Omni 的 Thinker-Talker 异步流水线、Fish S2 Pro 的 Dual-AR 串行嵌套、Ming Omni 的全模态输出这些具体的计算过程中, 一步步完成对共性的抽象, 边界清晰, 扩展灵活。

当然, 我们也远不完美, 跨节点的 multi-stage pipeline、更完整的 diffusion stage 支持、端到端的 RL 训练集成, 这些都在推进中。但有了清晰的 stage 抽象、统一的调度器接口、分层通信和跨 stage 显存预算, 这些特性自然应是从从容容游刃有余。我们期待下个阶段的 SGLang Omni, 功能完整, 模型生态友好。对于新生代的 multi-stage model, 不再需要从零开始搭一套 pipeline, 在十几个文件里到处填 if-else, 优雅地将模型拆分为调度流水段, 配合我们的 callback hook, 声明一份拓扑, 剩下的调度、通信、显存管理, 交给框架。我们期待这件事情做成, 也相信它能做成。

专多 stage 推理框架该怎么设计, 你也想和一群八一起, 真正完成一次工业级的工程训练——

我们非常欢迎你加入。不管你是做系统出身还是半路出家, 不管你擅长 kernel 优化还是调度逻辑, 期待你的加入。

编辑于 2026-05-30 08:41 · 美国


字节跳动自研大模型
豆包大模型
19.9元1000万tokens, 低至2.8折
[立即购买](#)



豆包大模型, 1000万tokens仅需19.9元, 每日限量...

豆包大模型, 字节跳动自研大模型, 具有更强的推理能力, 多模态理解能力, GUI操作能力和前端页面编程能力, 模型 ... [查看详情](#)

 火山引擎 的广告 ×



理性发言, 友善互动

4 条评论

默认 最新



ZHANG Mingxing

太成功了, 不用上课, 直接明年邀请你来分布式系统课 guest lecture 🤪

05-30 · 陕西

 回复  4



Chayenne Zhao 作者

有点鞭尸了章老师😂😂😂

05-30 · 美国

 回复  喜欢



风轮 

蛮辛苦的 看社区上的人都在加班修PR。可惜手里没卡, 不然也想试试。

05-30 · 福建

 回复  1



qzylalala

看了下 roadmap, 任务基本被 pick 完了😂

06-08 · 浙江

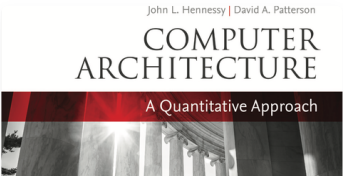
 回复  1

推荐阅读



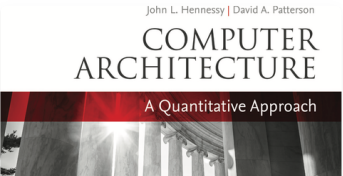
15年软件架构师经验总结: 在ML领域, 初学者踩过的5个坑

机器之心 发表于机器之心



第7章 领域专用架构 (二)

顾大壮



第7章 领域专用架构 (一)

顾大壮

什么是动态规划 (Dynamic Programming) ? 动态规划...

本系列的主旨是让复杂的问题变得通俗易懂。有些问题乍看起来可能非常吓人。如何才能真正地去理解问题的本质, 并在此基础上, 令复杂的计算机科学看起来有趣、易懂, 从而不会让人心生恐惧, 是...
直觉优先 发表于数据结构与 ...